Core Banking & Multi-Branch Transaction Ledger System

Complete Project Summary, Technical Architecture, Concurrency Controls, RBAC Security, and Presentation Guide

Technology Stack:	Java 17 (JDK LTS), HikariCP 5.1.0, PostgreSQL (Supabase Cloud), Gson 2.11, SLF4J + Logback, JUnit 5 + Mockito
Architecture:	Modular 4-Layer: Embedded REST API (JDK HttpServer) → RBAC Security Guard Proxy → Core Banking Engine → HikariCP DAO → PostgreSQL (DDL + PL/pgSQL)
Web Portal:	Single-Page Glassmorphism Portal (HTML5, Vanilla CSS3, Modern ES6+ JavaScript, 100% Zero-External-CDN UI runtime)
Test Verification:	70 automated unit tests (100% passing) covering concurrency ordering, overdraft prevention, RBAC matrices, and PBKDF2 hashing

1. Executive Summary & Core Value Proposition

Modern financial ledger platforms demand absolute consistency, deterministic transaction ordering, strict compliance auditing, and zero tolerance for money multiplication or deadlocks. The **Meridian Core Banking System** is an academic-grade, production-style banking platform designed from the ground up to solve complex enterprise banking challenges using lightweight Java 17 and PostgreSQL.

Zero-Loss Atomic Ledger	Deadlock-Free Two-Phase Locking	5-Tier RBAC & Step-Up MFA
All monetary transactions use rigorous BigDecimal arithmetic, manual JDBC transaction boundaries with rollback on any failure, and DB trigger auditing.	Transfers acquire row-level locks on both participant accounts ordered deterministically by primary key (min(A, B) then max(A, B)) to eliminate circular waits.	PBKDF2-HMAC-SHA256 password hashing with salt, 5 granular roles, brute-force account lockout (5 attempts), and step-up MFA for transfers ≥ ■10,000.

2. Team Division & Architectural Contributions

The system is partitioned into 4 distinct, cohesive modules reflecting industry separation of concerns:

Team Member / Role	Core Architectural Scope	Key Deliverables & Code Artifacts
Member 1 **Database Architect & DBA**	Relational schema modeling (3NF), declarative integrity constraints, stored procedures, audit triggers, and composite query indexing.	• sql/schema.sql (branches, customers, accounts, transactions) • process_fund_transfer stored procedure • trg_high_value_flag trigger (≥ ■10,000 audit) • B-Tree indexes on (sender, created_at) and customer fields

Member 2 **Core Banking Engine & Java Lead**	Database connection pooling singleton, domain entities, transaction orchestration, deadlock prevention algorithms, and domain exceptions.	• DatabaseConfig.java (HikariCP singleton pool, env/.env support) • BankingService.java (deposit, withdraw, transfer logic) • AccountDAO.java & TransactionDAO.java • InsufficientBalanceException, AccountNotFoundException • BankingServiceTest.java (unit test suite)
Member 3 **Security, RBAC & Audit Engine**	Cryptographic credential protection, role-based authorization guards, session context propagation, and forensic compliance auditing.	• PasswordHasher.java (PBKDF2-HMAC-SHA256, 65k iter, salt) • Role.java & Permission.java (5 roles, 8 permissions) • SecureBankingService.java (proxy decorator enforcing RBAC) • sql/security_schema.sql (bank_users, security_audit_log) • SecurityTest.java (RBAC, lockout, and crypto tests)
Member 4 **UI, REST API & Presentation Lead**	Lightweight embedded HTTP server, RESTful JSON routing, session authentication filters, and responsive single-page web application.	• ApiServer.java (JDK HttpServer, thread pool executor) • AuthFilter.java & SessionStore.java (Bearer token security) • Controllers: Auth, Account, Transaction, Customer, Audit • index.html, styles.css, app.js (Interactive SPA portal) • Step-up MFA verification modal & printable PDF receipt logic

3. Relational Database Schema & Storage Architecture

The relational schema is hosted on PostgreSQL (Supabase Cloud) and enforces strict ACID properties, foreign key cascade protections, and domain constraints at the database engine level.

Table Name	Primary Key	Key Attributes & Constraints	Foreign Keys & Indexes
branches	`branch_id (SERIAL)`	branch_name, ifsc_code (UNIQUE, NOT NULL), address	Referenced by accounts (ON DELETE RESTRICT)
customers	`customer_id (SERIAL)`	first_name, last_name, email (UNIQUE), phone (UNIQUE), national_id (UNIQUE)	B-Tree index on email, phone
accounts	`account_number (BIGSERIAL)`	account_type IN ('SAVINGS','CHECKING','LOAN'), balance NUMERIC(15,2) CHECK(≥0.00), status IN ('ACTIVE','FROZEN','CLOSED')	FK: customer_id, branch_id. Pessimistic row locking via FOR UPDATE
transactions	`transaction_id (BIGSERIAL)`	amount NUMERIC(15,2) CHECK(>0.00), transaction_type IN ('DEPOSIT','WITHDRAWAL','TRANSFER','INTEREST'), status, reference_note	FK: sender_account, receiver_account. Composite indexes on (sender, date) and (receiver, date)
audit_logs	`audit_id (BIGSERIAL)`	action_type, account_number, details, logged_at	Populated automatically by trigger trg_high_value_flag for tx ≥ ■10,000
bank_users	`user_id (SERIAL)`	username (UNIQUE), password_hash (PBKDF2), role, failed_attempts, is_locked, last_login_at	FK: customer_id (links online banking user to customer entity). Partial index on is_locked=TRUE
security_audit_log	`event_id (BIGSERIAL)`	event_type, username, account_number, details, ip_address, occurred_at	Immutable append-only audit trail for compliance and forensic investigations

Concurrency Control & Deadlock Prevention Protocol

The Problem: When Customer 1 transfers money to Customer 2 while Customer 2 simultaneously transfers to Customer 1, naive locking locks Account 1 first in Thread A and Account 2 first in Thread B, creating a classic circular deadlock condition.
The Meridian Solution: The Java service layer enforces deterministic lock acquisition ordering:

- long firstAcc = Math.min(fromAccount, toAccount);
- long secondAcc = Math.max(fromAccount, toAccount);

Both threads always acquire locks in the exact same numerical sequence (e.g. #1001 then #1002), mathematically proving the impossibility of circular wait.

4. Security Architecture & Role-Based Access Control (RBAC)

Security is implemented as a dedicated architectural layer rather than an afterthought. Every request passes through authentication filters, thread-local security context binding, and pre-invocation RBAC guards.

System Role	Primary Persona	Permitted Domain Operations	Security Restrictions
CUSTOMER	Alice Smith (#1) Bob Jones (#2)	• View own account balance & transaction history • Execute inter-account fund transfers • Step-up MFA verification	Restricted to own customer_id accounts. Cannot deposit cash or open accounts directly.
TELLER	Teller 01 (Branch Front Desk)	• Perform cash deposits into any customer account • Process counter cash withdrawals • Check account status and balances	Cannot freeze accounts, alter interest rates, view compliance audit logs, or open new branches.

BRANCH_MANAGER	Manager 01 (Executive)	• Approve and open new customer accounts • Freeze, unfreeze, and close accounts • Inspect branch transaction ledgers & audit events	Cannot delete customer profiles or alter core database tables.
AUDITOR	Auditor 01 (Risk & Compliance)	• Complete read-only view of security_audit_log • Inspect all transactions, high-value triggers ≥ ■10k • Review failed login attempts and lockout records	Strictly read-only. Cannot originate transfers, deposits, or alter account states.
ADMIN	System Admin	• Full administrative supervisory privileges • Unlock locked user accounts • System diagnostics & health telemetry	Audited by immutable security log for all actions.

5. REST API Architecture & Single-Page Banking Portal

The application features a lightweight, high-performance embedded REST API built using the standard JDK HttpServer (zero heavy servlet containers required). All responses are serialized to JSON with standard HTTP status codes.

HTTP Method & Endpoint	Required Role	Description & Request Payload	Response Status
`POST /api/auth/login`	**Public**	Authenticates username & password; issues 24-hour Bearer session token	200 OK / 401 Unauthorized
`POST /api/auth/mfa`	**Authenticated**	Verifies 6-digit one-time PIN for high-value transfer authorization	200 OK / 400 Bad Request
`GET /api/accounts`	**CUSTOMER+**	Returns accounts for active user or query customer ID	200 OK [JSON Array]
`POST /api/accounts`	**BRANCH_MANAGER+**	Opens new account: {customerId, branchId, accountType, initialBalance}	201 Created
`PATCH /api/accounts/{id}/status`	**BRANCH_MANAGER+**	Modifies status: ACTIVE, FROZEN, or CLOSED	200 OK
`POST /api/transactions/transfer`	**CUSTOMER+**	Atomic transfer: {fromAccount, toAccount, amount, note} (requires MFA if ≥ 10k)	200 OK / 402 / 403
`POST /api/transactions/deposit`	**TELLER+**	Cash deposit: {accountNumber, amount, note}	200 OK
`POST /api/transactions/withdraw`	**TELLER+**	Cash withdrawal with balance validation: {accountNumber, amount, note}	200 OK / 402 Insufficient
`GET /api/audit`	**AUDITOR, ADMIN, MGR**	Inspects chronological forensic audit logs with event type & user filters	200 OK
`GET /health`	**Public**	Live liveness probe: returns {"status":"UP"}	200 OK

Interactive Web Portal Modules (Single Page Application)

- **1-Click Demo Personas:** Instant credential pre-fill for Alice, Bob, Teller01, Manager01, Auditor01, and Admin.
- **Live Metrics Dashboard:** Real-time counters showing Active Accounts, Total Ledger Volume, Daily Velocity, and High-Value Audit Flags.
- **Move Money Terminal:** Unified deposit, withdrawal, and fund transfer forms with instant client balance validation and reference notes.
- **Step-Up MFA Challenge Modal:** Automatically engages when a user initiates a transfer of ■10,000 or greater.
- **Printable Transaction Receipts:** Automatically generates timestamped, reference-coded receipts upon successful fund transfer.
- **Forensic Audit Console:** Live feed of security events (logins, transfers, lockouts) with IP address tracking and event filtering.

6. Evaluation & Live Presentation Guide

Follow these concise instructions during the academic evaluation or viva presentation to showcase full project capabilities:

Presentation Phase	Terminal Command / User Action	What to Explain to the Examiners
Phase 1: Automated Test Suite	`mvn test`	Showcase 70 passing JUnit 5 tests. Highlight Mockito DAO mocks, concurrency ordering checks, and password hash verifications without needing an external DB.
Phase 2: Launch Platform	`java -jar target/core-banking-system-1.0.0.jar`	Explain automatic schema initialization (DatabaseInitializer), connection pool bootstrap (HikariCP), and demo user seeding (DemoUserSeeder).

Phase 3: Customer Fund Transfer	Open http://localhost:8080/ Sign in as **alice** (Customer #1)	Transfer ■1,000 to Bob (#1002). Demonstrate atomic balance updates, receipt generation, and two-phase lock ordering in backend logs.
Phase 4: Step-Up MFA Challenge	Transfer ■15,000 from Alice to Bob	Showcase how the system intercepts transfers ≥ ■10,000 and requires one-time MFA PIN verification before proceeding.
Phase 5: Teller Operations	Switch user to **teller01**	Perform counter cash deposit. Show that tellers cannot approve new accounts or view compliance audit logs.
Phase 6: Compliance Audit	Switch user to **audit01** Open *Compliance & Audit Trail*	Demonstrate immutable audit records showing previous transfers, high-value transaction flags (DB trigger), and user session timestamps.